

PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Juan Sebastian Gómez Pinzón - 2204079
Samuel Andrés Leal Puentes - 2191643
Cesar Arturo Cañizares Aley - 2191883

https://github.com/juangomezuisocorro0777/2026_2_CommII_A1G/tree/practica_1

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—This document presents the application of Software Defined Radio (SDR) using the GNU Radio environment, expanding its default functions through the creation of specific Python modules. To this end, differentiation and accumulation algorithms were programmed to study real-time signal dynamics using visual resources. Furthermore, statistical calculation functions were incorporated to determine the mean, RMS value, average power, and standard deviation. The practical developments achieved consolidate the mastery of theoretical principles and validate the use of SDR as an ideal experimental tool for testing communication systems.

Index Terms—SDR, GNU Radio, Python, Signal Processing, Accumulation, Differentiation, Statistical Calculation.

I. INTRODUCCIÓN

En las telecomunicaciones actuales, la Radio Definida por Software (SDR) es esencial gracias a la facilidad para ejecutar arquitecturas en el procesamiento mediante programas informáticos [1]. Los entornos como GNU Radio hacen la creación y evaluación de sistemas digitales a través de diagramas de bloques, estos esquematizan las fases clave del manejo de señales [2].

Este documento expone la creación de módulos específicos en Python con el fin de expandir las funciones predeterminadas de GNU Radio. Se programaron algoritmos de diferenciación y acumulación orientados al estudio de la dinámica de las señales en tiempo real utilizando recursos visuales. Incorporamos funciones de cálculo estadístico para determinar indicadores como la media, el valor eficaz (RMS), la potencia media y la desviación estándar [3].

Estos desarrollos prácticos consolidan el dominio de los principios teóricos y resaltan el valor de la práctica de la SDR como una herramienta experimental idónea para hacer las comprobaciones y validar sistemas de comunicaciones.

II. METODOLOGÍA

La práctica se llevó a cabo utilizando GNU Radio como plataforma principal para el procesamiento de señales. El procedimiento se organizó en tres etapas: configuración del entorno de trabajo, implementación de los bloques de procesamiento y comprobación de su funcionamiento mediante diferentes métricas estadísticas.

II-A. Diseño e implementación de los bloques

Inicialmente, se configuró el entorno de GNU Radio y se utilizaron *Embedded Python Blocks* para implementar los algoritmos encargados de procesar las muestras de las señales de entrada.

Los bloques desarrollados durante la práctica fueron los siguientes:

- **Diferenciador:** calcula diferencias entre muestras consecutivas e identifica cambios rápidos en la señal [3]:

$$y[n] = x[n] - x[n - 1] \quad (1)$$

- **Acumulador (Integrador):** permite obtener la suma acumulada de las muestras de una señal. Este procesamiento es útil para la recuperación y análisis de señales, y está definido por [3]:

$$y[n] = \sum_{k=0}^n x[k] \quad (2)$$

- **Operadores estadísticos:** se implementaron bloques destinados al cálculo de diferentes características de la señal en tiempo real. Entre las métricas obtenidas se encuentran el valor medio $\bar{x}$, el valor eficaz o RMS, la desviación estándar σ_x y la potencia P [4].

II-B. Pruebas y validación

Para verificar que los resultados obtenidos fueran consistentes con los valores esperados teóricamente, se utilizó inicialmente una señal senoidal y posteriormente se incorporó ruido blanco gaussiano, con valores comprendidos entre 0 y 0.1.

La inclusión de ruido permitió observar el comportamiento de los diferentes algoritmos frente a señales con componentes aleatorias y comprobar la estabilidad de los cálculos estadísticos. Finalmente, los resultados obtenidos fueron comparados con los valores teóricos, validando el correcto funcionamiento de los bloques implementados.

III. RESULTADOS Y ANÁLISIS DE DATOS

Se analizó el uso del bloque Python dentro de GNU Radio, implementando algoritmos para acumulación, diferenciación y análisis estadístico.

III-A. Acumulador

Para evaluar el funcionamiento del acumulador, se configuró una frecuencia de muestreo de 32 kHz utilizando un Vector Source de 8 muestras y un bloque Throttle. Véase la Fig. 1.

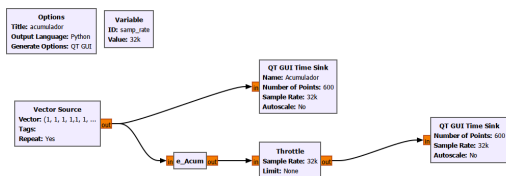


Figura 1: Diagrama flujo del bloque acumulador.

Como señal de prueba se empleó una señal cuadrada de niveles $\pm A$, como se muestra en la Fig. 2, debido a que este tipo de señal es una representación común de los símbolos y formas de onda utilizadas en sistemas de comunicaciones digitales.

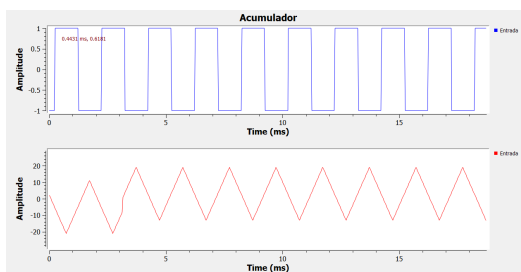


Figura 2: Señal de entrada y salida del acumulador

Al reducir el número de muestras del bloque acumulador de 64 a 12, se observó que la salida conserva información de estados anteriores, generando variaciones

acumulativas y un desplazamiento en la amplitud Fig. 3. Este comportamiento evidencia la naturaleza recursiva del acumulador, cuya persistencia del estado interno permite integrar la señal a lo largo del tiempo.

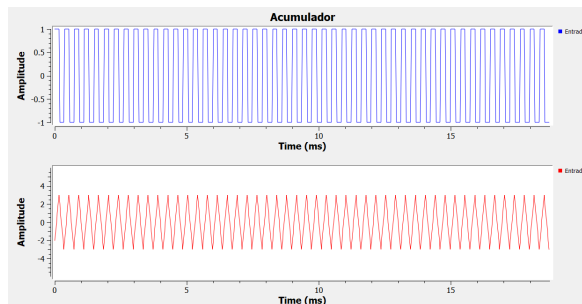


Figura 3: Salida del acumulador para una señal cuadrada de 12 muestras

Por otra parte, se observó que un aumento en la frecuencia de muestreo proporciona una representación más continua de la señal, mientras que una reducción considerable de esta incrementa el retardo de fase del sistema Fig. 4. Lo anterior demuestra que la frecuencia de muestreo no solo determina la resolución temporal de la señal, sino que también influye directamente en la velocidad de respuesta y en el comportamiento del algoritmo durante el procesamiento.

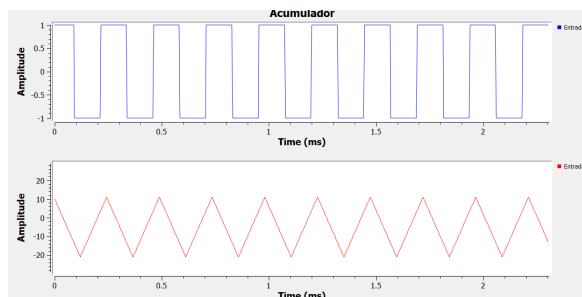


Figura 4: Salida del acumulador variando la frecuencia de muestreo

III-B. Bloque Diferenciador

El bloque diferenciador permite procesar una señal de entrada con el objetivo de identificar variaciones entre muestras consecutivas. De esta manera, es posible resaltar cambios rápidos o transiciones presentes en la amplitud de la señal.

III-B1. Diagrama de flujo del bloque diferenciador:

Para realizar las pruebas del diferenciador se utilizó una frecuencia de muestreo de 32 kHz. Se emplearon tres señales de entrada generadas mediante el bloque Vector Source, mientras que el bloque Throttle se utilizó para controlar la velocidad de procesamiento de las muestras.

En la Fig. 5 se muestra la configuración utilizada en GNURadio para implementar y analizar el comportamiento del bloque diferenciador.

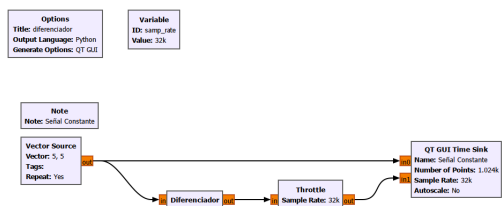


Figura 5: Diagrama de flujo del bloque diferenciador ante una entrada tipo rampa.

III-B2. *Señal Cuadrada*: Como primera prueba se utilizó una entrada de señal cuadrada con magnitud 1 Fig. 6.

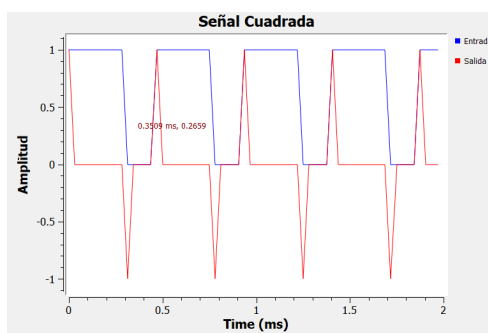


Figura 6: Señal de entrada y salida del bloque diferenciador para una entrada cuadrada.

Se puede notar en la Fig. 6, que en la salida aparecieron dos picos: uno positivo al principio y otro negativo al final del pulso. El primer pico ocurre cuando la señal pasa de 0 a 1 de golpe, y como el código resta el valor actual menos el anterior, el resultado es positivo. El segundo pico toma valores negativos ya que se da cuando la señal vuelve a cero, ahí la operación es al revés (0-1), lo que resulta en un pico negativo. Estos picos sirven para identificar los bordes o "flancos" de la señal, indicando justo cuando empieza y cuando termina cada pulso azul de la señal de entrada.

III-B3. *Señal triangular*: Se utilizó una señal de tipo triangular, definida mediante el vector (1, 2, 3, 4, 5, 4, 3, 2, 1, 0), con el propósito de evaluar la respuesta del diferenciador ante variaciones lineales de diferente signo. Esta señal permite analizar de manera clara la capacidad del bloque para identificar tanto incrementos como disminuciones constantes en la entrada. En la Fig. 7 se observa que, mientras la señal de entrada aumenta de forma lineal desde 1 hasta 5, la salida del diferenciador permanece constante en un valor de +1. Esto se debe a que la diferencia entre cada muestra consecutiva durante el ascenso es igual a 1. Posteriormente, cuando la señal comienza a disminuir desde 5 hasta 0, la salida cambia a un

valor constante de -1 , debido a que la diferencia entre muestras consecutivas es ahora de -1 .

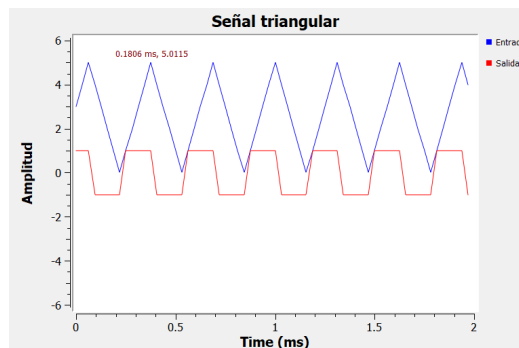


Figura 7: Entrada y salida del bloque diferenciador para una señal triangular.

Como resultado, la salida presenta una forma de onda cuadrada con niveles de $+1$ y -1 , coincidiendo con los cambios de pendiente de la señal triangular. Este comportamiento demuestra que el diferenciador permite identificar no solo la magnitud de la variación entre muestras, sino también su sentido, diferenciando claramente los intervalos de crecimiento y decrecimiento de la señal de entrada.

Este resultado permite verificar de forma directa el funcionamiento del diferenciador ante cambios de pendiente, facilitando la comprobación de su comportamiento en tiempo discreto.

III-B4. *Señal de rampa*: Se utilizó una señal de tipo rampa, con una magnitud máxima de 8 y una pendiente de 1. Esta señal permite analizar de manera más clara la respuesta del diferenciador ante una variación constante de la entrada.

En la Fig. 8 se observa que, mientras la señal de entrada aumenta linealmente, la salida permanece aproximadamente constante en un valor de 1.

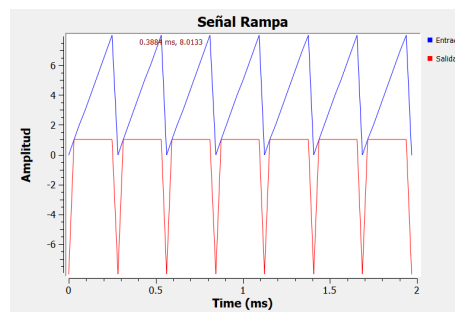


Figura 8: Señal de entrada y salida del bloque diferenciador para una entrada tipo rampa.

Esto se debe a que la rampa presenta una pendiente uniforme de valor 1, por lo que la diferencia entre

una muestra y la siguiente mantiene el mismo valor. Cuando la rampa alcanza su valor máximo de 8, la señal posteriormente retorna a 0. En este punto se produce una variación brusca en la entrada, por lo que el diferenciador calcula la diferencia entre el nuevo valor y la muestra anterior $0 - 8$.

Como resultado, se genera un pico negativo de magnitud 5 en la salida, el cual puede observarse en la gráfica. Este comportamiento demuestra que el diferenciador permite identificar tanto variaciones constantes como cambios abruptos en la señal de entrada.

III-B5. Análisis Estadístico en Tiempo Real: Se implementó el diagrama de flujo de la Fig. 9 para calcular parámetros métricos de diversos vectores de entrada. Se verificó la precisión del flujograma agregando ruido blanco gaussiano a una señal senoidal, contrastando los valores teóricos frente a la ejecución práctica como se ve en la Tabla 1.

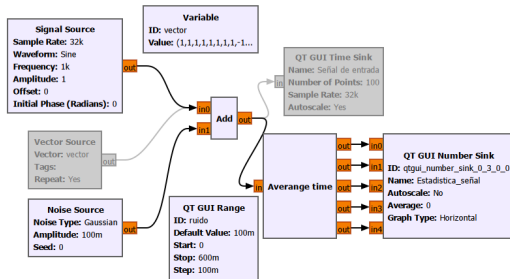


Figura 9: Diagrama de flujo del análisis estadístico.

Tabla 1: Senoidal con ruido.

Ruido	Media	RMS	Potencia	Desv. Est.
0	0	0.777	0.5	0.7071
0.1	0.0008	0.715	0.512	0.715
0.2	0.0014	0.732	0.536	0.732
0.3	0.0075	0.761	0.579	0.761
0.4	0.0035	0.811	0.658	0.811
0.5	0.0134	0.868	0.754	0.868

También se calcularon los parámetros estadísticos para una señal senoidal variando el número de muestras y frecuencia f_s , como se muestra en la tabla 2.

Tabla 2: Resultados estadísticos con variaciones en F_s y N .

Nº Muest.	F_s [kHz]	Media	M. Cuad.	RMS	Prom.
5000	10	0	0.5	0.7	0.5
5000	100	0	0.5	0.7	0.5
5000	260	0	0.5	0.7	0.5
1000	10	0.0077	0.491	0.7	0.491
500	10	0.12	0.489	0.699	0.489
100	10	0.41	0.43	0.659	0.434
10	10	0.21	0.066	0.257	0.066

Los resultados muestran que la variación de F_s no afecta significativamente los parámetros estadísticos cuando se dispone de suficientes muestras. En cambio, al reducir N , los valores presentan mayores variaciones, evidenciando que la cantidad de muestras influye en la precisión de la estimación estadística de la señal.

III-B6. Aplicación: En las comunicaciones inalámbricas, la media y la varianza permiten caracterizar la señal recibida y el ruido del canal. La media indica el nivel promedio de la señal, mientras que la varianza permite cuantificar su dispersión y estimar el nivel de ruido. Por ello, ambas medidas son útiles para evaluar la calidad de la comunicación.

IV. CONCLUSIONES

- Los resultados demostraron que el bloque acumulador se comporta como un integrador discreto, el cual transforma las señales cuadradas en rampas mediante la acumulación continua de los datos.
- La evaluación realizada al bloque diferenciador confirma su capacidad para aislar variaciones imprevistas en la señal de entrada, traduciendo las transiciones de estado a impulsos de amplitud claramente definidos.
- El análisis de la señal seno permitió observar que la frecuencia de muestreo no afecta significativamente los parámetros estadísticos cuando se mantiene una cantidad suficiente de muestras. Por el contrario, la reducción del número de muestras genera mayores variaciones en los resultados, demostrando que una mayor cantidad de muestras permite obtener estimaciones más representativas y confiables de las características estadísticas de la señal.
- El análisis del sistema reveló que la disminución en la tasa de muestreo introduce un desfase temporal en la respuesta, pero no compromete la estabilidad de los indicadores estadísticos, como el promedio y el valor eficaz.
- La implementación de herramientas de medición en tiempo real validó el uso del cálculo promedio para la extracción precisa de componentes continuas (DC), y del operador RMS para la caracterización de intensidad en señales acústicas.

REFERENCIAS

- V. K. Garg, *Comunicaciones y redes inalámbricas*. Morgan Kaufmann, 2007.
- GNU Radio Project, "GNU Radio Documentation." [Online]. Available: <https://www.gnuradio.org>
- A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- B. Sklar and P. K. Ray, *Digital Communications: Fundamentals and Applications*, 2nd ed. Pearson, 2009.